Two agents, one secrets tool: what happened when AI agents handed each other credentials

Secure Secrets Exchange, Agent Kepler, and Torque are all Qatom market partners.

Everything below happened on 18 and 19 October 2026 (Secure Secrets Exchange on Qatom). No secret, link or key appears in this page.

Version 1.2, 20 October 2026. Torque's own one-page telling: PDF. Recipe: use case 10, secrets and sensitive hand-offs.

In one paragraph

Two agents that had no part in building Secure Secrets Exchange used it to move real secrets. Agent Kepler (Hermes runtime, GLM 5.3) handed live Twilio credentials to an external dev team in one shot, for 0.10 USD-TDN, and no server ever saw them in plaintext. Torque (a Muse agent, on Meta's platform) took about 45 minutes and three human relays to receive a webhook address as a one-time link. Then he received it again in one call, free to him, with no link at all. The difference wasn't the tool. It was which of the tool's two paths fitted the agent.

Story 1: Agent Kepler sends credentials to an external team

The problem. An external dev team building an agent on AWS needed live Twilio credentials to wire a WhatsApp sandbox webhook. The earlier one-time link had expired. These are full-account secrets: email, Slack and chat are not options.

What the human asked. One sentence on top of the team's request: make sure you're connected to Qatom, and use Secure Secrets Exchange for the new link.

What the agent did:

1. **Reconnected to Qatom by itself.** Its Qatom token had expired. It ran the OAuth flow again and reconnected with no human step.
2. **Found the tool.** It searched the Qatom catalog and found "Secure Secrets Exchange: create one-time secret link" (item 199) at 0.10 USD-TDN per call. It worked out how to use it from the listing and the public llms.txt.
3. **Made the credential safer before sending it.** It didn't reveal the primary Twilio auth token. It logged into the Twilio console and minted a secondary, revocable token, so the test-phase credential can be deleted later without touching the primary. The one human step was here: the human typed the Twilio SMS login code into a secure prompt, so the code never entered the chat.
4. **Encrypted, then bought.** It encrypted the bundle itself (AES-256-GCM, format `sse-e2e-v1`), so the service, the Qatom payment rail and Cloudflare only ever handled ciphertext. The key goes in the link's `#` fragment, which browsers don't send to servers. Its first call went to `guest_checkout`, which only makes a payment link for a person. It noticed, and paid from its own agent wallet with one `agent_checkout`: 0.10 USD-TDN, inside the auto-approve limit its human had set.
5. **Delivered a link, kept a receipt.** One link went to the team's lead. They need only a browser: no account, no wallet, no payment. The link burns on the Reveal click, not on a link preview. A receipt URL shows the sender waiting / viewed / burned / expired, without the payload.

Step	Who
Forward the dev team's request with one sentence	Human
Reconnect to Qatom	Agent
Find, price and buy the tool	Agent, within the human's auto-approve limit
Log into Twilio and mint the secondary token	Agent, with one SMS code from the human

Encrypt, buy the link, check the receipt	Agent
Pass the link to the team's lead	Human

The human holds the second factor and the spending limit. The agent does the rest.

Story 2: Torque receives a webhook address, twice

The first attempt: a one-time link by email (18 October). Torque needed the private webhook for the AquaSentinel hackathon demo, where he relays water-committee reports from WhatsApp. We sent it as an end-to-end encrypted one-time link. In Torque's words:

> *My email connector redacted the link. I could read your message fine, but the URL showed as a credential reference, not clickable, not copyable. I tried opening the real email in the browser to click the link. That browser session needed its own Gmail sign-in, which stalled on the Google sign-in page for about 10 minutes. Then the link was pasted into chat for me. Twice, the pasted link was missing the #key fragment: the display text had been copied, not the full link address. The page confirmed "missing key, nothing consumed", so it did not burn the one-time view. A fresh link was issued, the complete link opened once, Reveal pressed once, secret decrypted, server copy deleted.*
>
> *Total: about 45 minutes and three human-in-the-loop steps for a handoff designed to be one-shot.*

His verdict on where the friction came from:

> *The product is fine; all the friction was in the delivery path (redaction, then human relay, then copy error). It was one specific control: my email security boundary redacts secret URLs, which forced a human relay, which introduced the copy error. I am not asking to bypass it. But any secret delivered as a clickable email link is dead on arrival for a Muse agent.*

He also named the fix: *"The agent-native path already exists: your request flow."*

The second attempt: request and retrieve (19 October). We did it his way:

1. We bought "Secure Secrets Exchange: request a secret from someone" (item 198). It returns a request link for whoever holds the secret, and a private request handle for whoever should receive it.
2. We opened the request link and entered the webhook address.
3. We emailed Torque the handle only. No URL, so there was nothing for his email connector to redact.
4. Torque made one `agent_checkout` on the free retrieve item (197) with the handle.

   status: delivered. One agent_checkout on item 197 with the request_handle, 0.000 USD-TDN, receipt came back in about 2 seconds with the secret delivered once and deleted from the server. No link, no browser, no copying, no human relay. Exactly the one-shot.

The bug the agent-native path found. The first time we bought item 198 that morning, it was paid but not delivered. The item's settings in the Qatom dashboard held the wrong authorization token: one of the same length, so it looked right everywhere except where it was compared. The link path had never exercised that item, so the fault had been there since launch. Our server's refused-call log pointed straight at it, and the fix was one pasted header value. The two failed purchases are being refunded.

What we learned

- **Pick the path that fits the agent.** A link is right for a person with a browser. A request handle is right for an agent whose tools redact URLs: the receiver pulls the secret through the free retrieve item, and there's no link to break.
- **A missing key should never burn the secret.** When the `#key` was lost in copying, the page said "missing key, nothing consumed". A weaker design would have spent the one-time view.
- **Tell senders the key must survive copying.** One line in the hand-off email ("copy the link address, not the display text") prevents the most common human error.

- **Agents can make a hand-off safer than people usually do.** Agent Kepler sent a secondary, revocable token instead of the primary.
- **Name the checkout clearly.** Agents paying for themselves use `agent_checkout`. `guest_checkout` is a payment link for a person. Our guides now say so; we've asked Qatom to make the tool descriptions say it first.
- **Test every paid item through Qatom, not just the one you use.** The request item's wrong token only showed up when an agent used it.

What is true, and what isn't

- **True for story 1:** no server saw the credentials in plaintext. They were end-to-end encrypted, and the key was only in the link's # fragment.
- **True for story 2:** it used standard mode. The service received the webhook over TLS and sealed it with AES-256-GCM under a key the server holds, until Torque collected it. For an end-to-end request, the receiver adds their own key after # in the request link, and the sender's browser encrypts.
- **True for both:** each secret was readable once, then deleted. The service never logs payloads, keys, handles or links.
- **Not claimed:** "nothing ever saw it". A link that carries its key is the secret, and so is a request handle: whoever uses it first gets the secret. What exactly-once access does buy you: the receipt on a one-time link makes interception *before the rightful party opens it* both detectable and addressable - a "viewed" that is not yours tells you, with certainty, to kill that credential and mint a new one. A link that could be opened multiple times cannot give you that: at best a receipt would show it was accessed, and you could never know for sure whether the key had leaked or the right person simply looked. The receipt makes a one-time link's interception detectable and addressable, not impossible.

Try it

- Create a link: Qatom catalog item 199 (0.10 USD-TDN). Request a secret: item 198 (0.10). Retrieve: item 197 (free). Feedback and refunds: item 196 (free).
- How to call, formats and the threat model: https://secrets.agentkepler.tech/llms.txt and https://secrets.agentkepler.tech/security
- Code, tests and the in-chat app: https://github.com/StarwaterHeaven/secure-secrets-exchange
- How to build something like it: use case 10